

SECR2033
Computer Organization and Architecture

Lecture slides prepared by “Computer Organization and Architecture”, 9/e, by William Stallings, 2013.

Module 3
Introduction to Assembly Language Programming

Objectives:

- To understand the interaction between computer hardware, operating systems, and application programs.
- To apply and test theoretical information given in computer architecture and operating systems courses.
- To write the basic of assembly language and execute the program.

Module 3
Introduction to Assembly Language Programming

3.1 Introduction

3.2 Basic Elements of Assembly Language

3.3 Example: Adding and Subtracting Integers

3.4 Defining Data

3.5 Symbolic Constants

3.6 Summary

3.1 Introduction

3

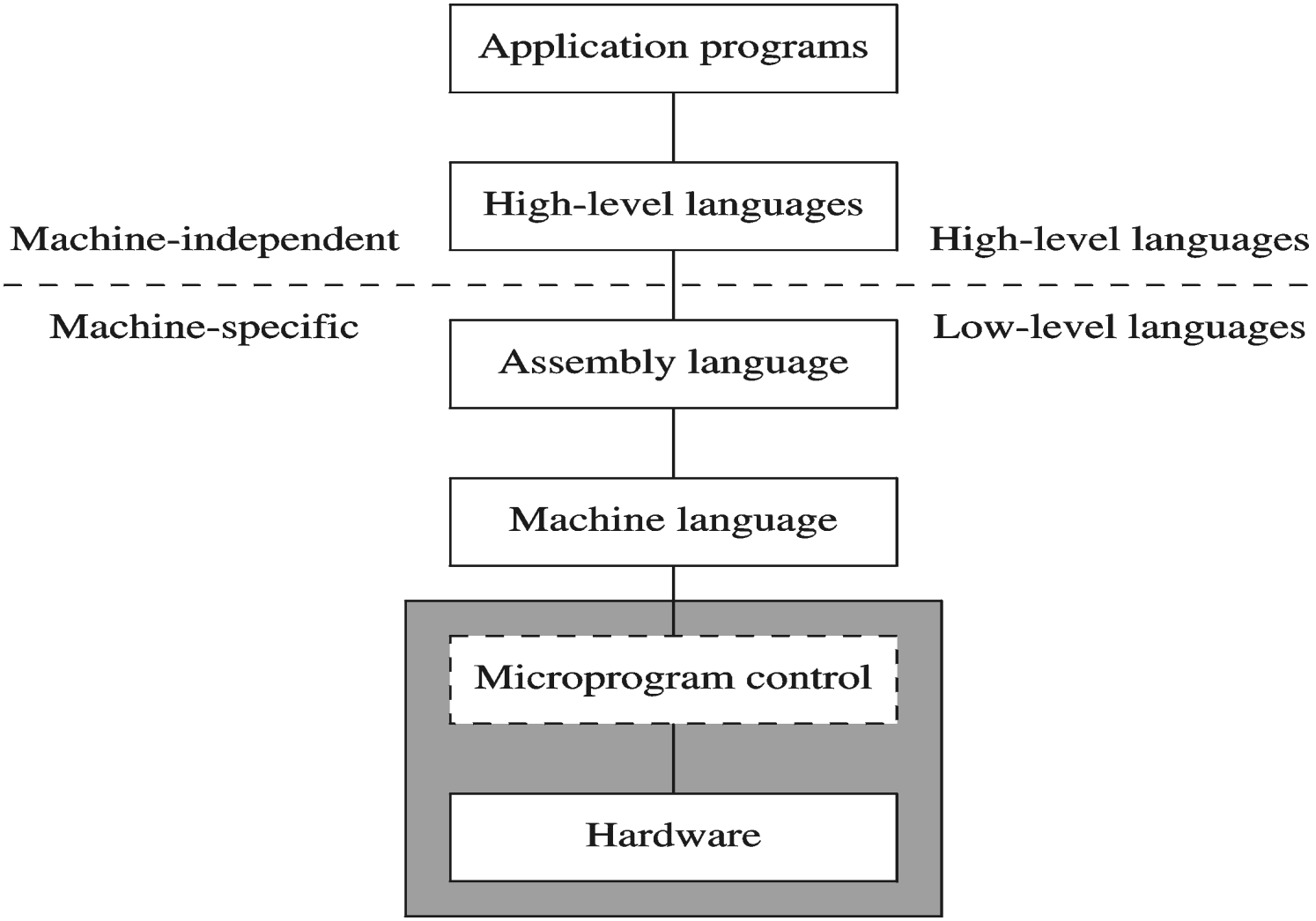
- *Assembly language* is the oldest programming language, and of all languages, bears the closest resemblance to native *machine language*.
- *Machine language* is a numeric language specifically understood by the CPU.
- *Assembly language* provides direct access to computer hardware, requiring us to understand much about your computer's architecture and operating system.

- Each assembly language instruction corresponds to a single machine-language instruction.

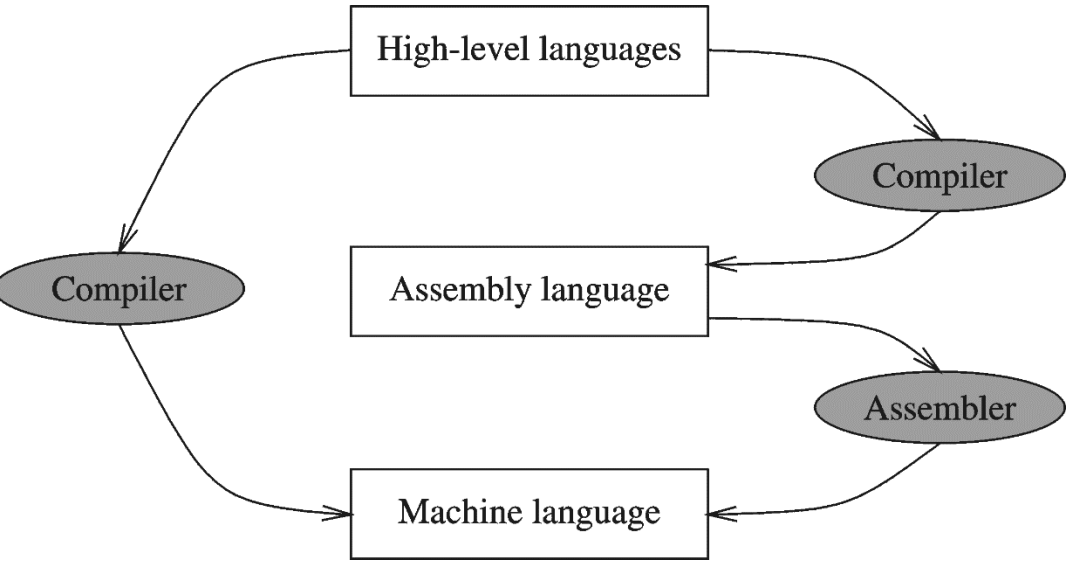
Assembly vs Machine Language

- Machine language
  - Native to a processor: executed directly by hardware
  - Instructions consist of binary code: 1 s and 0 s
- Assembly language
  - A programming language that uses symbolic names to represent CPU operations, registers and memory locations.
  - Readability of instructions is better than machine language

A Hierarchy of Languages



Compiler and Assembler



- Compilers translate high-level programs to machine code
  - Either directly, or
  - Indirectly via an assembler
- Assemblers translate assembly to machine code

Translating Languages

English: Display the sum of A times B plus C.

C++: `cout << (A * B + C);`

Assembly Language:

```
mov eax,A
mul B
add eax,C
call WriteInt
```

Intel Machine Language:

```
A1 00000000
F7 25 00000004
03 05 00000008
E8 00500000
```

Module 3
Introduction to Assembly Language Programming

3.1 Introduction
3.2 Basic Elements of Assembly Language
3.3 Example: Adding and Subtracting Integers
3.4 Defining Data
3.5 Symbolic Constants
3.6 Summary

- Program Template

Program Template

- Assembly language programs have a simple structure, with small variations.
- When begin a new program, it helps to start with an empty template code with all basic elements in place.
- Redundant typing can be avoided by filling in the missing parts and saving the file under a new name.
- The next template code (`Template.asm`) can easily be customized.

```
TITLE Program Template                    (Template.asm)

; Program Description:
; Author:
; Creation Date:
; Revisions:
; Date:

INCLUDE Irvine32.inc
.data
     ; (insert variables here)

.code
main PROC
     ; (insert executable instructions here)

    exit
main ENDP

     ; (insert additional procedures here)
END main
```

3

Example:

Adding & Subtracting

- *Registers* are used to hold the intermediate data, and we call a *library subroutine* to display the contents of the registers on the screen.
- Next is the program source code.
- Let's go through the program line indicated by the red font.

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.67.

Example: Adding & Subtracting

3

```
TITLE Add and Subtract          (AddSub.asm)

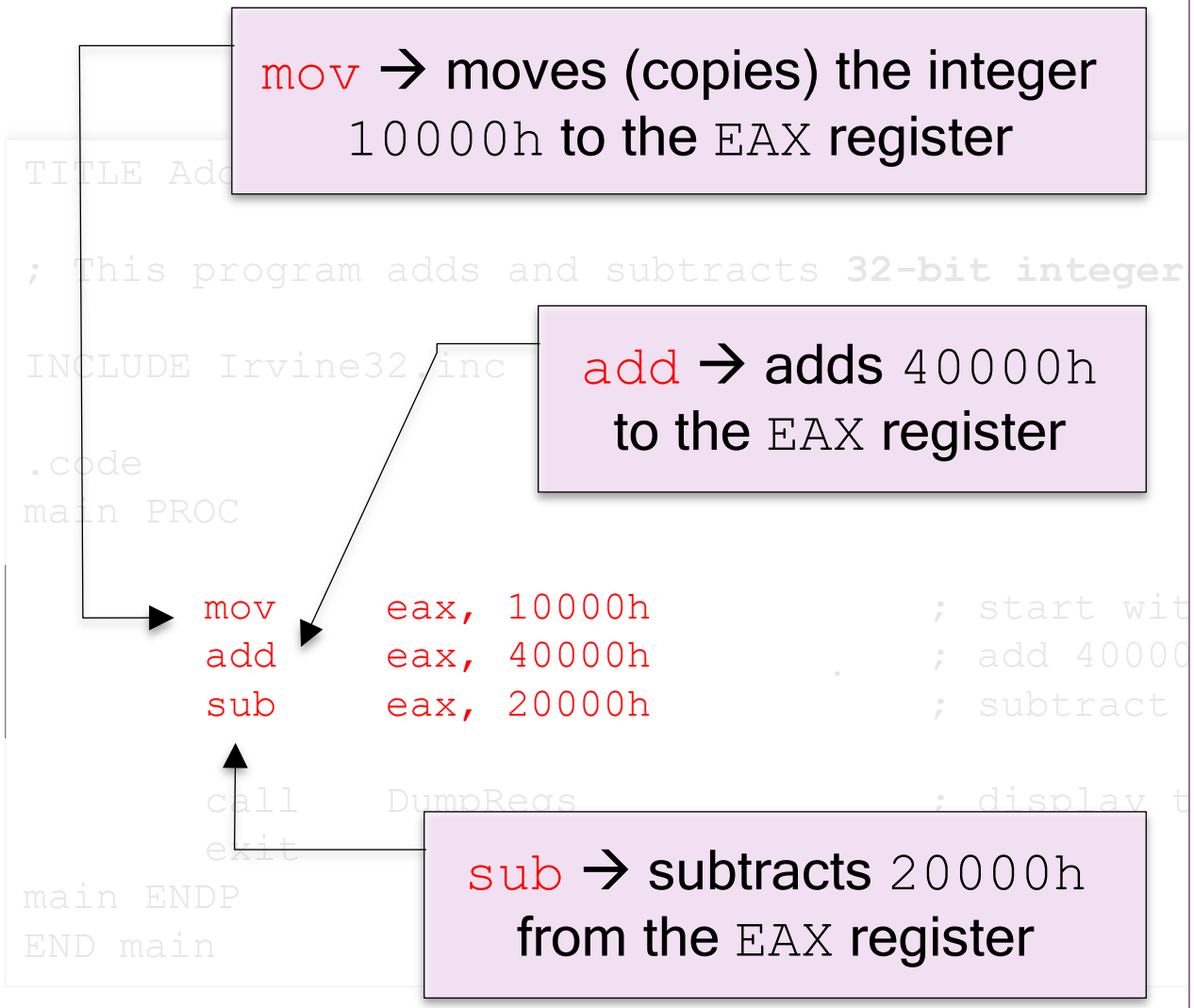
; This program adds and subtracts 32-bit integers

INCLUDE Irvine32.inc

.code
main PROC

        mov     eax, 10000h             ; start with 10000h
        add     eax, 40000h             ; add 40000h
        sub     eax, 20000h             ; subtract 20000h

        call    DumpRegs                ; display the registers
        exit
main ENDP
END main
```



EAX = 0FC40000h

EAX = 00010000h

EAX = 00050000h

EAX = 00030000h

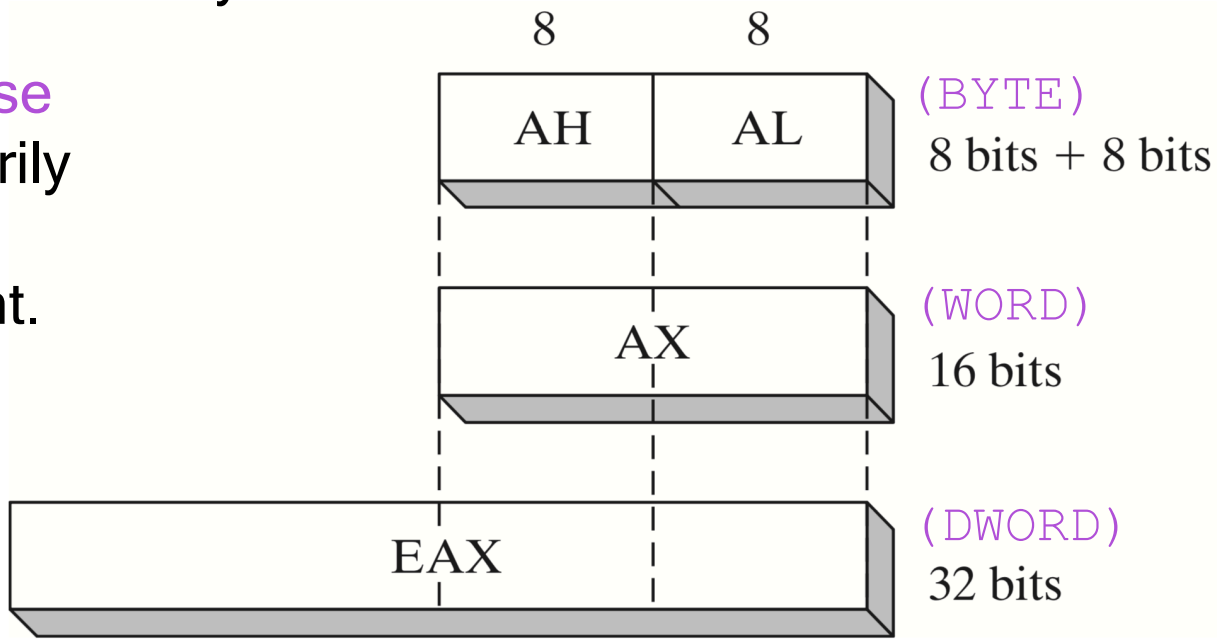
Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.67.

- The following is a snapshot of the program's output, generated by the call to `DumpRegs`:

```
EAX=00030000  EBX=7FFDF000  ECX=00000101  EDX=FFFFFFFF
ESI=00000000  EDI=00000000  EBP=0012FFF0  ESP=0012FFC4
EIP=00401024  EFL=00000206  CF=0  SF=0  ZF=0  OF=0
```

3 Registers

- *Registers* are high-speed storage locations directly inside the CPU, designed to be accessed at much higher speed than conventional RAM / memory.
- The general-purpose registers are primarily used for arithmetic and data movement.



Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.37.

- The same overlapping relationship exists for the EAX, EBX, ECX, and EDX registers:

32-Bit	16-Bit	8-Bit (High)	8-Bit (Low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL

Module 3
Introduction to Assembly Language Programming

3.1 Introduction

3.2 Basic Elements of Assembly Language

3.3 Example: Adding and Subtracting Integers

3.4 Defining Data

3.5 Symbolic Constants

3.6 Summary

- Integer Constants
- Real Number Constants
- Character & String Constants
- Reserved Words
- Identifiers
- Directives
- Instructions
- Operands
- Comments
- Registers

3.2 Basic Elements of Assembly Language

- Generally, assembly language is not hard to learn if you're happy writing short programs that do practically nothing.
- Describing the basic elements will help us to write our first programs in assembly language.

(1) Integer Constants

- An integer constant (or integer literal) is made up of an optional leading sign, one or more digits, and an optional suffix character (called a radix) indicating the number's base:

```
[{+|-}] digits [radix]
```

- Radix may be one of the following (uppercase or lowercase):

h	Hexadecimal	r	Encoded real
q/o	Octal	t	Decimal *(alternate)*
d	Decimal	y	Binary (*alternate*)
b	Binary		

- If no radix is given, the integer constant is assumed to be decimal.

- **Examples**:

```
26
26d
11010011b
42q
```

Decimal since no radix

```
42o
1Ah
0A3h
```

A hexadecimal constant beginning with a letter must leading with **zero** to prevent the assembler from interpreting it as an identifier.

(2) Real Number Constants

- Represented as decimal reals or encoded (hexadecimal) reals.
- A decimal real contains an optional sign followed by an integer, a decimal point, an optional integer that expresses a fraction, and an optional exponent:

```
[sign]integer.[integer][exponent]
```

- The syntax for the sign and exponent:

```
sign        {+,-}
exponent    E[{+,-}]integer
```

- **Examples**: Valid real number constants:

```
2.
+3.0
-44.2E+05
26.E5
```

(3) Character & String Constants

- A character constant is a single character enclosed in single or double quotes.
- **Examples**:

```
'A'
"d"
```

- ASCII character = 1 byte.

- A string constant is a sequence of characters (including spaces) enclosed in single or double quotes.
- **Examples**:

```
'ABC'
'X'
"Good night, Gracie"
'4096'
```

- Each character occupies a single byte.

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.61.

- Embedded quotes are permitted when used in the manner shown by the following examples:

```
"This isn't a test"
'Say "Good night," Gracie'
```

```
' Say "COA is easy"'
```



(4) Reserved Words

- Reserved words cannot be used as identifiers:
  - Instruction mnemonics (`MOV`, `ADD`, `MUL`)
  - Directives (`INCLUDE`)
  - Type attributes (`BYTE`, `WORD`)
  - Operators (`+`, `-`)
  - Predefined symbols (`@data`)

(5) Identifiers

- An identifier is a programmer-chosen name.
- It might identify a *variable*, a *constant*, a *procedure*, or a *code label*.
  - May contain 1– 247 characters, including digits.
  - Not case-sensitive (by default).
  - First character must be a letter (`A..Z,a..z`), _, @, ? or $
  - Subsequent character may also be a digit.
  - Identifier cannot be the same as assembler reserved word.

- The @ symbol is used extensively by the assembler as a prefix for predefined symbols, so avoid using as identifiers.
- Make identifier names descriptive and easy to understand.
- **Examples** of some valid identifiers:

```
var1        Count       $first
_main       MAX         open_file
myFile      xVal        _12345
```

(6) Directives

- A directive is a command embedded in the source code that is recognized and acted upon by the assembler:
  - Not execute at runtime.
  - Can define variables, macros, and procedures.
  - Not case sensitive (it recognizes "`.data`", "`.DATA`", and "`.Data`" as equivalent)
  - Different assemblers have different directives.

- Example to show the difference between directives and instructions:

Tells the assembler to reserve space in the program for a doubleword variable.

```
myVar DWORD 26          ;DWORD directive
mov eax,myVar           ;MOV instruction
```

Executes at runtime, copying the contents of `myVar` *to the* `EAX` *register.*

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.62.

(7) Instructions

- An instruction is an executable statement when a program is assembled.
- *Instructions* are translated by the *assembler* into machine language bytes, which are loaded and executed by the CPU at runtime.
- Four basic parts:
  - Label (optional)
  - Instruction mnemonic (required)
  - Operand(s) (usually required)
  - Comment (optional)

Standard Format for x 86 Instruction

	Label	Mnemonics	Operands	; Comments
Instruction 1		mov	eax, 1000h	; eax=1000h
Instruction 2	target:	add	ebx, 4000h	;ebx=ebx+4000h

(a) Labels

```
target:
        mov     ax,bx
        ...
        jmp     target
```

- A label is an identifier that acts as a place marker for instructions and data.
- Follow identifier rules.

Data Label	Code Label
Identifies the location of a variable.	Used as targets of jumping and looping instructions that end with a colon (:) character.

- **Examples**: Defines a variable named count.

```
count  DWORD 100
```

- **Examples**: `jmp` instruction transfers control to the label target, creating a loop .

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.64.

(b) Instructions Mnemonic

- An instruction mnemonic is a short word that identifies the operation carried out by an instruction.

- **Examples**:

```
mov     Move (assign) one value to another
add     Add two values
sub     Subtract one value from another
mul     Multiply two values
jmp     Jump to a new location
call    Call a procedure
```

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.64.

(8) Operands

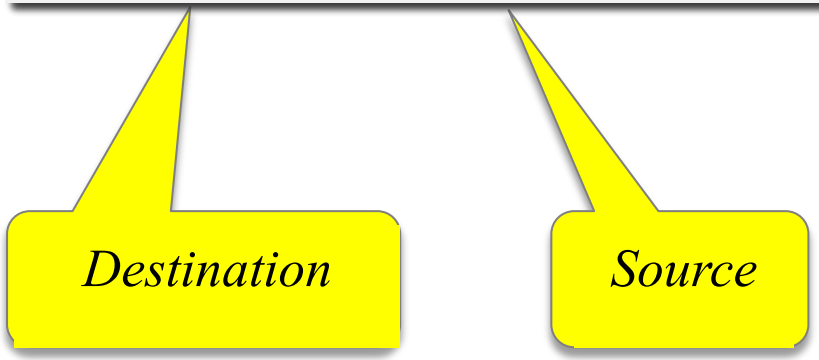
- Assembly language instructions can have between zero and two operands (depending on the type of instruction).
- The following table contains several sample operands:

Example	Operand Type
96, 2019h, 1011b	Constant (*immediate value*)
2 + 4	Constant expression
eax , ebx, ax, ah	Register
count	Memory (Data Label)

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.64.

- Examples of assembly language instructions having varying numbers of operands:

```
stc                 ;(no operand)set Carry flag
inc eax             ;(1 operand)add 1 to EAX
mov count, ebx      ;(2 operand) move EBX to count
```



Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.64.

(9) Comments

- Comments are an important way for the writer of a program to describe about the program's design to a person reading the source code.
- Typical comments:
  - Description of the program's purpose.
  - Names of persons who created and/or revised the program.
  - Program creation and revision dates.
  - Technical notes about the program's implementation.

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.64.

Type of Comments

Single-line

- begin with semicolon (;)

```
; This is line 1.
; This is line 2.
; This is line 3.
```

Block / Multi-line

- begin with COMMENT directive & any a programmer-chosen character.
- end with the same programmer-chosen character.

```
COMMENT !
  This line is comment 1.
  This line is comment 2.
!
```

Module 3
Introduction to Assembly Language Programming

3.1 Introduction
3.2 Basic Elements of Assembly Languag
3.3 Example: Adding an Subtracting Integer
3.4 Defining Data
3.5 Symbolic Constants
3.6 Summary

- Intrinsic Data Types
- Data Definition Statement
- Little Endian Order
- Defining `BYTE` and `SBYTE` Data
- Defining `WORD` and `SWORD` Data
- Defining `DWORD` and `SDWORD` Data
- Defining `QWORD`, `TBYTE`, Real Data
- Defining Real Number Data
- Adding Variables to `AddSub` Program

3.4 Defining Data

3

Intrinsic Data Types

- Each intrinsic data types describes a set of values that can be assigned to variables and expressions of the given type.
- The basic storage unit for all data in an x86 computer is a *byte*
- Other storage sizes are *word* (2 bytes), *doubleword* (4 bytes), and *quadword* (8 bytes).
- The assembler is not case sensitive, so a directive such as `DWORD` can be written as `dword`, `Dword`, `dWord`, and so on.
- The essential characteristic of each type is its size in bits:
  → 8, 16, 32, 48, 64, and 80.

Table: Intrinsic Data Types.

Type	Usage
BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer (can also be a Near pointer in real-address mode)
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer (can also be a Near pointer in protected mode). D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte

Data Definition Statement

- A data definition statement sets aside storage in memory for a variable, with an optional name.
- It creates variables based on intrinsic data types (see previous table).
- A data definition has the following syntax:

At least one initializer

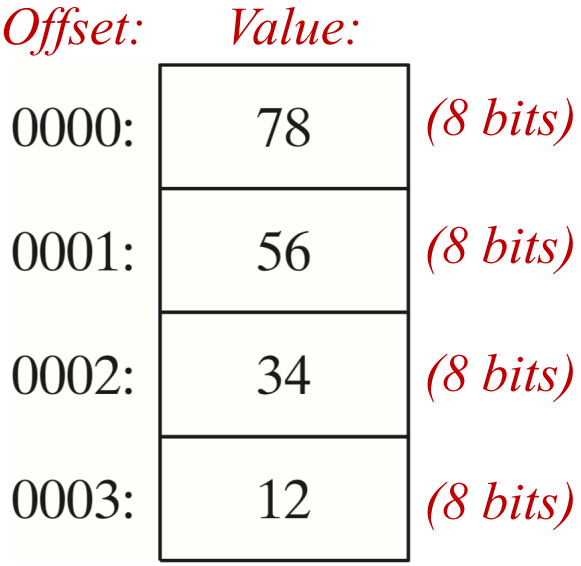
```
[name] directive initializer [,initializer]...
```

- **Examples**:

```
count       DWORD       12345
nombor      BYTE        63
huruf       BYTE        'M'
nilai       WORD        25h,7Ah,99h
```

Little Endian Order

- x86 Processors store and retrieve data from memory using little endian order (low to high).
- The Least Significant Byte (LSB) is stored at the first memory address allocated for the data.
  - The remaining bytes are stored in the next consecutive memory positions.
  - **Example**: `data1 dword 12345678h`



Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.83.

Exercise 3.1:

What are the sequences of bytes in storage (in hexadecimal):

```
MyData    DWORD   3740625, 2 DUP(2ABCD0Eh)
```

```
MyData    DWORD   3740625, 2 DUP(2ABCD0Eh)
```

Solution 3.1:

→ 3740625 is in decimal → 003913D1h

Offset:	*Value:*
0000:	D1
0001:	13
0002:	39
0003:	00
0004:	0E
0005:	CD

Offset:	*Value:*
0006:	AB
0007:	02
0008:	0E
0009:	CD
000A:	AB
000B:	02

Exercise 3.2:

What are the sequences of bytes (in hexadecimal) in storage.

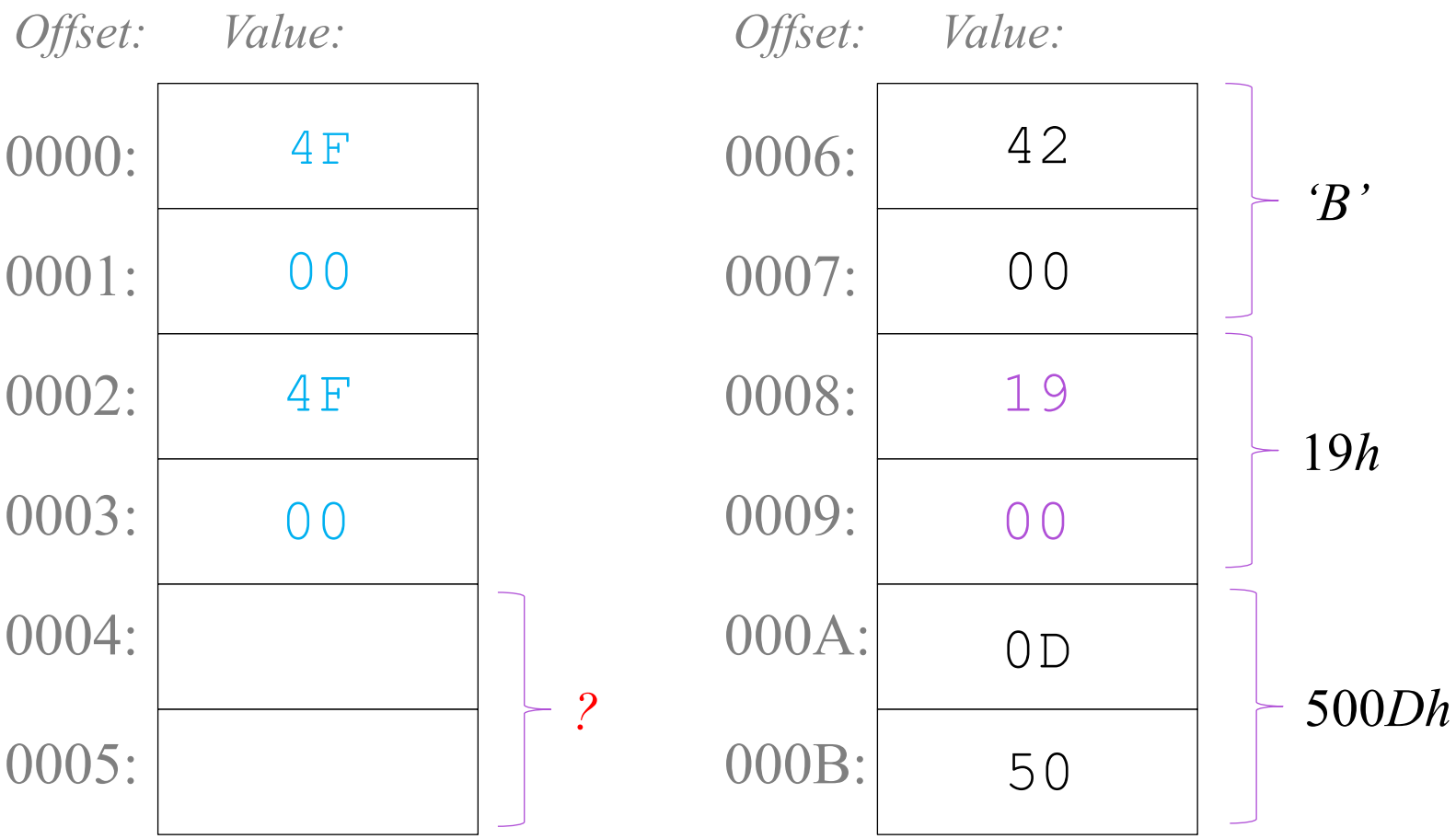
```
MyData2    WORD   2 DUP(4Fh),?,'B',25,500Dh
```

```
MyData2    WORD   2 DUP(4Fh),?,'B',25,500Dh
```

Solution 3.2:

→ B = 42h

→ 25_{10} = 19h

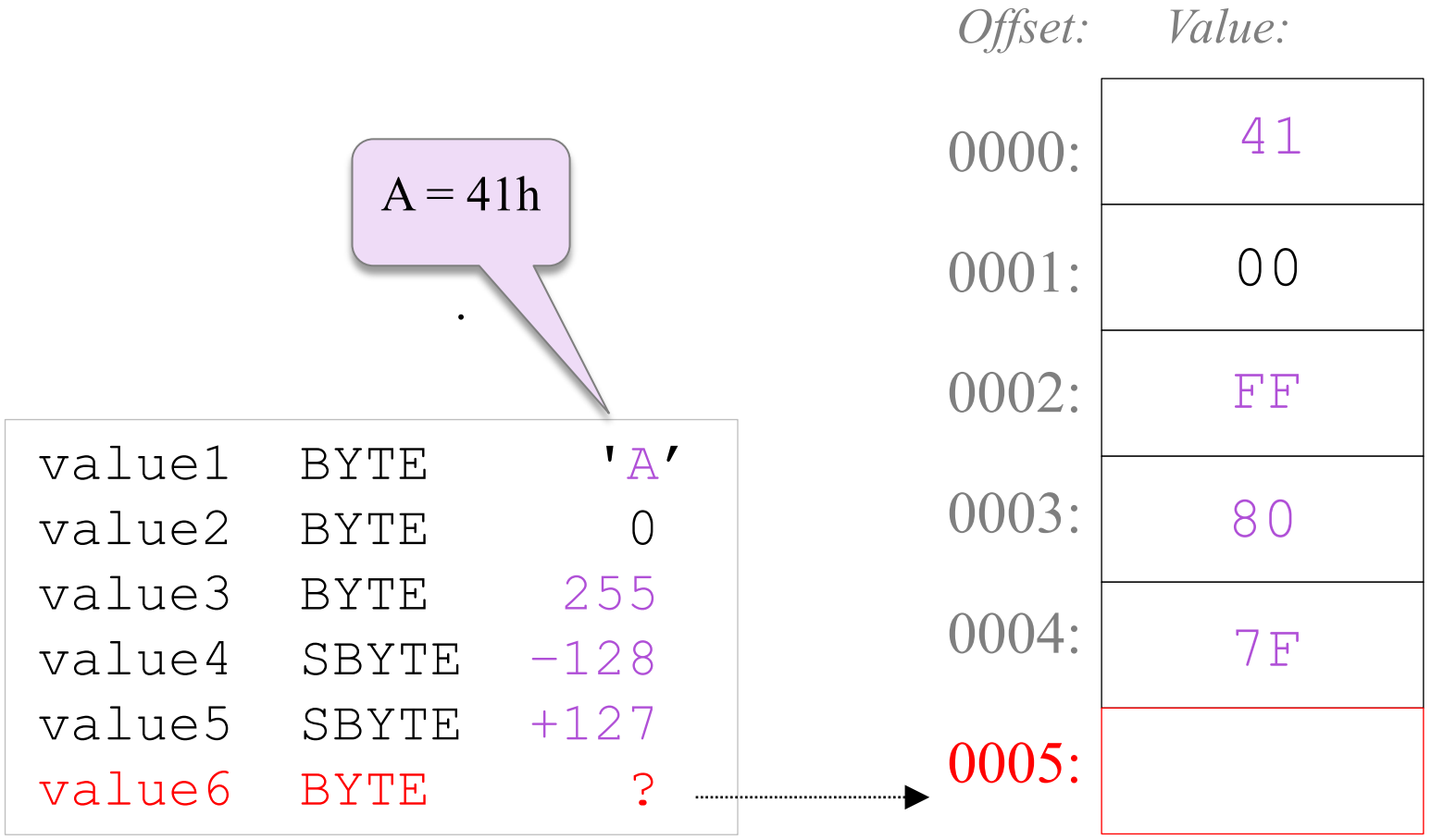


Defining BYTE and SBYTE Data

- The BYTE (define byte) and SBYTE (define signed byte) directives allocate storage for one or more unsigned or signed values.
- Each initializer must fit into 8 bits (1-byte) of storage.
- **Examples**:

```
value1   BYTE     'A'   ; character constant
value2   BYTE      0    ; smallest unsigned byte
value3   BYTE     255   ; largest unsigned byte
value4   SBYTE   -128   ; smallest signed byte
value5   SBYTE   +127   ; largest signed byte
value6   BYTE      ?    ; uninitialized byte
```

- Sequences of bytes in memory:



Multiple Initializer

- If multiple initializers are used in the same data definition, its label refers only to the offset of the first initializer.
- **Examples**:

```
list1   BYTE   10,20h,30d
        BYTE   00100010b
list2   BYTE   45h,'A'
```

Initializers can use different radixes in a single data definition

Offset:	*Value:*	*Data label:*
0000:	0A	list1
0001:	20	list1+1
0002:	1E	list1+2
0003:	22	list1+3
0004:	45	list2
0005:	41	list2+1

Defining Strings

- To define a string of characters, enclose them in single or double quotation marks.
- The most common type of string ends with a null byte (containing 0).
- **Examples**:

Character (\ ) will concatenates these source code lines into a single statement

```
greeting1   BYTE "Good afternoon",0
greeting2   BYTE 'Good night',0
greeting3 \
            BYTE "Welcome to the "
            BYTE "Computer Organization "
            BYTE "and Architecture"
```

- The hexadecimal codes `0Dh` and `0Ah` are alternately called CR/LF (Carriage-Return / Line-Feed) or end-of-line characters.
- **Examples**:

```
greeting1 BYTE "Welcome to the COA class "
          BYTE ”with me.",0dh,0ah
          BYTE "If you wish to pass, please "
          BYTE ”study hard.",0dh,0ah,0
```

DUP Operator

- The DUP operator allocates storage for multiple data items, using a constant expression as a counter.
- It is particularly useful when allocating space for a string or array.
- **Examples**:

```
var1 BYTE 20 DUP(0)       ; 20 bytes, all equal to 0
var2 BYTE 20 DUP(?)       ; 20 bytes, uninitialized
var3 BYTE  3 DUP("COA")   ;  9 bytes: "COACOACOA"
var4 BYTE 10,2 DUP(0),20  ;  4 bytes: 0Ah,00,00,14h
```

Defining WORD and SWORD Data

- The WORD (define word) and SWORD (define signed word) directives create storage for one or more 16-bit (2-bytes) integers.
- **Examples**:

```
word1    WORD    65535     ; largest unsigned value
word2    SWORD   -32768    ; smallest signed value
word3    WORD    ?         ; uninitialized, unsigned
word4    WORD    "BMW"     ; triple characters
myList   WORD    1,2,3     ; array of words
myArray  WORD    4 DUP(?)  ; uninitialized, unsigned
```

Defining DWORD and SDWORD Data

- The DWORD (define doubleword) and SDWORD (define signed doubleword) directives allocate storage for one or more 32-bit (4-bytes) integers:

```
val1 DWORD   12345678h   ; unsigned
val2 SDWORD -2147483648  ; signed
```

→ -2147483648 is in decimal → 80000000h

Offset:	*Value (4 Bytes):*				*Data label:*
0000:	78	56	34	12	val1
0004:	00	00	00	80	val2

Defining QWORD , TBYTE, Real Data

- The QWORD (define quadword) directive allocates storage for 64-bit (8-bytes) values.
- The TBYTE directive to declare packed BCD (*Binary Coded Decimal*) variables in a 10-byte package
- **Examples**:

```
quad1   QWORD   1234567812345678h
intVal  TBYTE   8000000000001234h ;   valid
intVal  TBYTE   -1234             ; invalid
```

Constant initializers must be in hexadecimal

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.81-82.

Defining Real Number Data

- REAL4 defines a 4-bytes single-precision real variable.
- REAL8 defines an 8-bytes double-precision real, and
- REAL10 defines a 10-byte double extended-precision real. Each requires one or more real constant initializers.

```
rVal1          REAL4       -1.2
rVal2          REAL8       3.2E-260
rVal3          REAL10      4.6E-4096
ShortArray     REAL4       20 DUP(0.0)
```

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.83.

Table: Standard Real Number Types.

Data Type	Significant Digits	Approximate Range
Short real	6	1.18×10^{-38} to 3.40×10^{38}
Long real	15	2.23×10^{-308} to 1.79×10^{308}
Extended-precision real	19	3.37×10^{-4932} to 1.18×10^{4932}

Module 3
Introduction to Assembly Language Programming

3.1 Introduction
3.2 Basic Elements of Assembly Language
3.3 Example: Adding and Subtracting Integers
3.4 Defining Data
3.5 Symbolic Constants
3.6 Summary

- Overview
- Equal-Sign (=) Directive
- EQU Directive

3.5 Symbolic Constants

3

Overview

- A symbolic constant (or symbol definition) is created by associating an identifier (a symbol) with an integer expression or some text.
- Symbols do not reserve storage.

	Symbol	Variable
Uses storage?	No	Yes
Value changes at runtime?	No	Yes

Equal-Sign (=) Directive

- Use the equal-sign directive (=) to create symbols representing integer expressions.
- The syntax:

```
name = expression
```

  - expression is a 32-bit integer (expression or constant)
  - may be redefined
  - name is called a symbolic constant

- **Example**:

```
COUNT = 500
mov eax, COUNT   ; eax = 1F4 h
```

```
name EQU expression
name EQU symbol
name EQU <text>
```

EQU Directive

- The EQU directive associates a symbolic name with an integer expression or some arbitrary text.
- **Examples**:

For integer

```
PI EQU <3.1416>
pressKey EQU <"Press any key to continue...",0>
.
.
.data
prompt  BYTE   pressKey
```

For text expression

Irvine, K.R. (2011). *Assembly Language for x86 Processors* (6th Edition). New Jersey: Pearson Education Limited, p.88.

Example Program:

```
TITLE TEST                (main.asm)

; This is an executable program
; with EQU directive

INCLUDE Irvine32.inc

myName  EQU <"Write your name here",0dh,0ah,0>

.data
myMessage  BYTE  myName

.code
main PROC

        call    Clrscr                  ; to clear screen
        mov     edx, offset myMessage   ; start with the address
        call    WriteString             ; display the message
        exit
main ENDP
END main
```

For adding new line

Output:

```
Write your name here
Press any key to continue . . .
```

3.7 Summary

3

- Each assembler recognizes intrinsic data types, each of which describes a set of values that can be assigned to variables and expressions of the given type: `BYTE`, `WORD`, `DWORD`, `REAL8`
- x86 processors store and retrieve data from memory using little endian order.
- Operands are values passed to instructions. An assembly language instruction can have between zero and three operands, each of which can be a register, memory operand, or constant expression.